

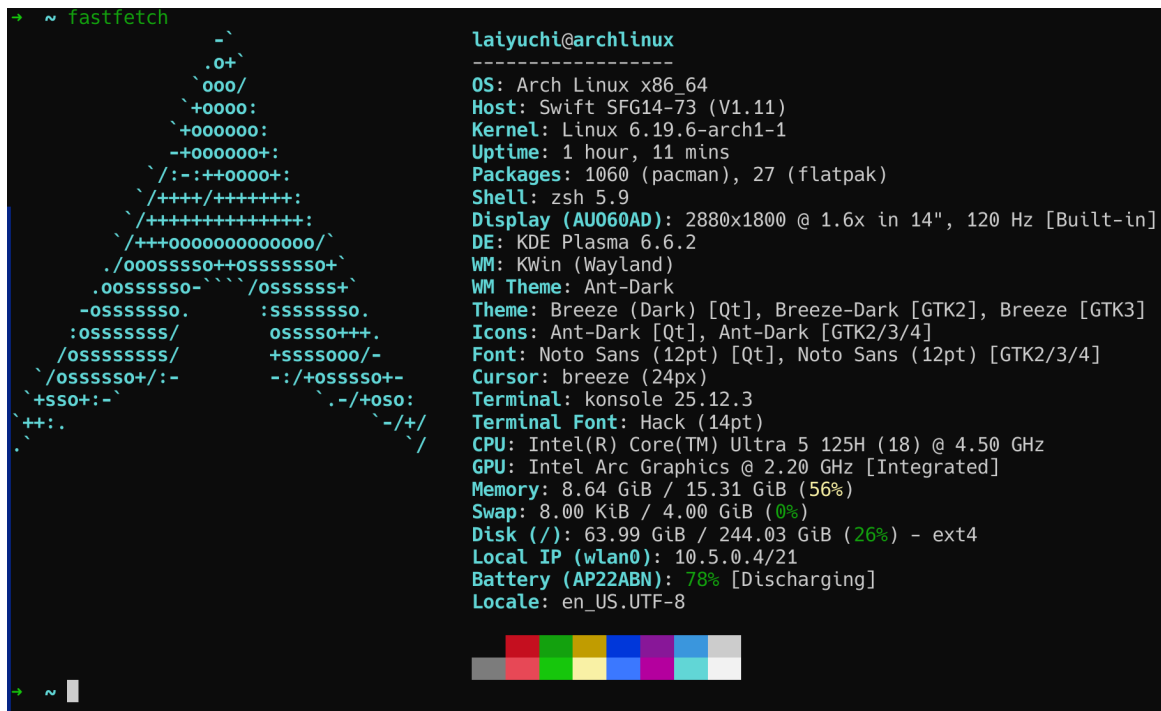
Numerical Method - Homework 3

B13902022 LAI, YU-CHI

1 Environment

1.1 OS info

I use Arch Linux as the OS, the hardware information is described as the following image.



```
→ ~ fastfetch

      .o+
     'ooo/
    +oooo:
   +oooooo:
  -+ooooooo+:
 /:-:++oooo+:
/++++/+++++:
/+++++++:
/+++++ooooooo/
./ooooSSSSo++oSSSSSo+`
 .oSSSSSo-````/oSSSS+`
-oSSSSSo.      :SSSSSo.
 :oSSSSS/      oSSSSo+++.
 /oSSSSSS/      +SSSSooo/-
`/oSSSSSo+/-:-  -:/+oSSSSo+-
++:-           -./+oso:
.              -/+//
               -/+//

laiyuchi@archlinux
-----
OS: Arch Linux x86_64
Host: Swift SFG14-73 (V1.11)
Kernel: Linux 6.19.6-arch1-1
Uptime: 1 hour, 11 mins
Packages: 1060 (pacman), 27 (flatpak)
Shell: zsh 5.9
Display (AU060AD): 2880x1800 @ 1.6x in 14", 120 Hz [Built-in]
DE: KDE Plasma 6.6.2
WM: KWin (Wayland)
WM Theme: Ant-Dark
Theme: Breeze (Dark) [Qt], Breeze-Dark [GTK2], Breeze [GTK3]
Icons: Ant-Dark [Qt], Ant-Dark [GTK2/3/4]
Font: Noto Sans (12pt) [Qt], Noto Sans (12pt) [GTK2/3/4]
Cursor: breeze (24px)
Terminal: konsole 25.12.3
Terminal Font: Hack (14pt)
CPU: Intel(R) Core(TM) Ultra 5 125H (18) @ 4.50 GHz
GPU: Intel Arc Graphics @ 2.20 GHz [Integrated]
Memory: 8.64 GiB / 15.31 GiB (56%)
Swap: 8.00 KiB / 4.00 GiB (0%)
Disk (/): 63.99 GiB / 244.03 GiB (26%) - ext4
Local IP (wlan0): 10.5.0.4/21
Battery (AP22ABN): 78% [Discharging]
Locale: en_US.UTF-8
```

Figure 1: The result of fastfetch (It could be installed through `pacman -S fastfetch`)

1.2 Setup

In all following implementations and tests, the multi-threading is **disabled** (see lines 1-5)

```
1 sudo pacman -S intel-oneapi-mkl, openblas, cblas
2 export MKL_NUM_THREADS=1
3 export OMP_NUM_THREADS=1
4 export OPENBLAS_NUM_THREADS=1
5 source /opt/intel/oneapi/setvars.sh
6
7 cd ~
8 git clone https://github.com/flame/blis.git
9 cd blis/
```

```

10 ./configure --enable-cblas auto
11 make
12 sudo make install
13 export BLIS_NUM_THREADS=1
14 cd ../code/
15
16 gcc -I /usr/local/include/blis my_steepest_descent.c -O2 -Wall -Wextra -std=gnu11 -o
  ↪ my_steepest_descent /usr/local/lib/libblis.a -lm
17
18 gcc my_steepest_descent.c -O2 -Wall -Wextra -std=gnu11 -lmkl_rt -lm -o my_steepest_descent
19
20 gcc my_steepest_descent.c -O2 -Wall -Wextra -std=gnu11 -lopenblas -lm -o my_steepest_descent
21
22 gcc my_steepest_descent.c -O2 -Wall -Wextra -std=gnu11 -lcblas -lm -o my_steepest_descent
23
24 curl -LsSf https://astral.sh/uv/install.sh | sh
25 source $HOME/.local/bin/env
26 uv venv matrix
27 source matrix/bin/activate
28 uv pip install scipy, numpy

```

- Line 1-5: Setup the environment for Intel oneMKL, OpenBLAS and CBLAS
- Line 7-14: Build the environment for AMD BLIS (according to the document of this [repo](#))
- Line 16: How I compile my code and link cblas.h to BLIS implementation
- Line 18: How I compile my code and link cblas.h to Intel oneMKL implementation
- Line 20: How I compile my code and link cblas.h to openblas implementation
- Line 22: How I compile my code and link cblas.h to netlib/CBLAS implementation
- Line 24-28: Setting up the virtual environment for the installation of SciPy.

2 Code

2.1 Steepest Descent Method

Usage (after compilation): `./my_steepest_descent n`, where integer `n` is the size of the matrix. After running, it will write `A`, `x` and `b` of the linear equation $Ax = b$ into the text file of same name.

```

1 // my_steepest_descent.c
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <math.h>
5 #include <time.h>
6 #include <string.h>
7 #include "cblas.h"
8 #define tol 1e-8

```

```

9  #define seed 53
10
11 double random_0_to_1() {
12     return (double)rand() / (double)RAND_MAX;
13 }
14
15 void generate_spd(double *A, int n) {
16     double *M = (double *)malloc((n*n) * sizeof(double));
17     for (int i = 0; i < n*n; i++) M[i] = random_0_to_1();
18     cblas_dgemm(CblasRowMajor, CblasTrans, CblasNoTrans, n, n, n, 1.0, M, n, M, n, 0.0, A,
19         ↪ n);
20     for (int i = 0; i < n; i++) A[i*n+i] += (double)n;
21     free(M);
22 }
23
24 void generate_vector(double *v, int n) {
25     for (int i = 0; i < n; i++) v[i] = random_0_to_1();
26 }
27
28 void steepest_descent(double *A, double *b, double *x, int n) {
29     double *r = (double *)malloc(n * sizeof(double));
30     double *Ar = (double *)malloc(n * sizeof(double));
31     double *Ax = (double *)malloc(n * sizeof(double));
32
33     memset(x, 0, n*sizeof(double));
34     cblas_dcopy(n, b, 1, r, 1);
35     double eps = cblas_dnorm2(n, r, 1);
36     while (eps > tol) {
37         cblas_dgemv(CblasRowMajor, CblasNoTrans, n, n, 1.0, A, n, r, 1, 0.0, Ar, 1);
38         double rr = cblas_ddot(n, r, 1, r, 1);
39         double rAr = cblas_ddot(n, r, 1, Ar, 1);
40         double alpha = rr / rAr;
41         cblas_daxpy(n, alpha, r, 1, x, 1);
42         cblas_dgemv(CblasRowMajor, CblasNoTrans, n, n, 1.0, A, n, x, 1, 0.0, Ax, 1);
43         cblas_dcopy(n, b, 1, r, 1);
44         cblas_daxpy(n, -1.0, Ax, 1, r, 1);
45         eps = cblas_dnorm2(n, r, 1);
46     }
47     free(r);
48     free(Ar);
49     free(Ax);
50 }
51
52 void write_vector(char *path, double *v, int n) {
53     FILE *fp = fopen(path, "w");
54     for (int i = 0; i < n; i++) {
55         fprintf(fp, "%f\n", v[i]);
56     }
57     fclose(fp);
58 }

```

```

59
60 void write_matrix(char *path, double *A, int n) {
61     FILE *fp = fopen(path, "w");
62     for (int i = 0; i < n; i++) {
63         for (int j = 0; j < n; j++) {
64             fprintf(fp, "%f%c", A[i*n+j], " \n"[j==n-1]);
65         }
66     }
67     fclose(fp);
68 }
69
70 int main(int argc, char **argv) {
71     int n = 100;
72     if (argc > 1) {
73         n = atoi(argv[1]);
74     }
75     srand(seed);
76
77     double *A = (double *)malloc((n*n) * sizeof(double));
78     double *b = (double *)malloc(n * sizeof(double));
79     double *x = (double *)malloc(n * sizeof(double));
80
81     generate_spd(A, n);
82     generate_vector(b, n);
83     steepest_descent(A, b, x, n);
84
85     double *Ax = (double *)malloc(n * sizeof(double));
86     cblas_dgemv(CblasRowMajor, CblasNoTrans, n, n, 1.0, A, n, x, 1, 0.0, Ax, 1);
87
88     write_matrix("A", A, n);
89     write_vector("x", x, n);
90     write_vector("b", b, n);
91
92
93 }

```

2.2 Correctness Checker

Usage: `python solution_checker.py`. After execution, the program will read A and b from the text file generated previously, and then calculate the solution x in $Ax = b$, print it out in the end.

```

1  # solution_checker.py
2  import numpy as np
3  from scipy.linalg import solve
4
5  A = np.loadtxt('A', dtype=np.float64)
6  b = np.loadtxt('b', dtype=np.float64)
7
8  x = solve(A, b)

```

9 `print(x)`

3 Result

3.1 Correctness of my code

```
(matrix) → code git:(main) × ./my_steepest_descent 10
(matrix) → code git:(main) × cat x
-0.027524
0.002931
0.044055
0.006838
-0.017339
0.047884
0.032042
0.009130
0.034176
-0.011189
(matrix) → code git:(main) × python solution_checker.py
[-0.02752364 0.00293079 0.04405479 0.0068383 -0.01733928 0.04788444
 0.03204178 0.00912986 0.03417578 -0.01118883]
(matrix) → code git:(main) ×
```

Figure 2: Using the checker on $n = 10$ case (openblas implementation, but other implementation is also correct), we can see our result is correct, align with the results from SciPy.

3.2 Performance on different matrix sizes

We use the built-in `time` command in linux to calculate the Real (Elapsed), User (user space) and Sys (kernel) time. The command is `time ./my_steepest_descent n`.

Size (N)	Library	Real (s)	User (s)	Sys (s)
100	Intel MKL	0.19	0.03	0.07
	OpenBLAS	0.06	0.01	0.04
	Netlib CBLAS	0.01	0.01	0.00
	AMD BLIS	0.05	0.01	0.04
500	Intel MKL	0.24	0.22	0.02
	OpenBLAS	0.23	0.21	0.01
	Netlib CBLAS	0.69	0.63	0.01
	AMD BLIS	0.27	0.20	0.01
1000	Intel MKL	1.87	1.79	0.04
	OpenBLAS	1.40	1.36	0.03
	Netlib CBLAS	5.08	5.04	0.02
	AMD BLIS	1.48	1.44	0.03
4000	Intel MKL	191.29	189.53	0.42
	OpenBLAS	259.38	256.78	0.43
	Netlib CBLAS	334.64	332.70	0.31
	AMD BLIS	206.97	204.60	0.35

Table 1: Execution Time Comparison of BLAS Libraries for `my_steepest_descent`

We can see that the intel oneMKL and AMD blis performs much better than openblas and netlib/CBLAS when n is large ($n = 4000$). The implementation from netlib is the worst one with respect to execution time.

4 Analysis

In High-Performance Computing (HPC), the performance gap between Vendor-Optimized (Intel MKL, AMD AOCL), Optimized Open-Source (OpenBLAS, BLIS), and Reference Implementations (Netlib CBLAS) is driven by three primary architectural optimizations.

1. The Memory Wall & Cache Blocking

The primary bottleneck in dense linear algebra is moving data from main memory to the CPU. Matrix multiplication requires $\mathcal{O}(n^3)$ operations on only $\mathcal{O}(n^2)$ data. High-performance libraries solve this using **multi-level cache blocking**. They slice matrices into specific blocks sized perfectly to fit into the processor's L1, L2, and L3 caches, as well as the Translation Lookaside Buffer (TLB). Netlib fails here because it uses naive nested loops, causing devastating cache misses.

- **Reference:** *Anatomy of High-Performance Matrix Multiplication* by Kazushige Goto and Robert van de Geijn (2008). This paper defines the “Goto Algorithm,” which forms the foundational cache-blocking strategy for GotoBLAS and OpenBLAS.

2. Hand-Tuned Assembly & SIMD Vectorization

Modern CPUs feature Single Instruction, Multiple Data (SIMD) units (e.g., AVX-512, NEON) that process multiple floating-point numbers per clock cycle. To maximize throughput, the innermost loops (“micro-kernels”) cannot be written in standard C; they must be written in hand-tuned assembly. Intel MKL and AMD BLIS leverage proprietary knowledge of their silicon to write cycle-perfect assembly. Netlib relies purely on compiler auto-vectorization, which is highly inefficient for complex matrix workloads.

- **Reference:** *BLIS: A Framework for Rapidly Instantiating BLAS Functionality* by Field G. Van Zee and Robert A. van de Geijn (2015). This paper explicitly details how BLIS isolates hardware-specific assembly into a tiny micro-kernel to achieve MKL-like performance.

3. Smart Multithreading and NUMA Awareness

Optimized libraries are explicitly designed for modern multi-core and Non-Uniform Memory Access (NUMA) architectures. Intel MKL binds threads strictly to physical cores (avoiding hyperthreading bottlenecks) and ensures memory is processed by the physically closest CPU socket. Netlib is strictly single-threaded and lacks parallel execution capabilities natively.

- **Reference:** *ScaLAPACK Users' Guide: Poor Performance (Netlib)*. The official documentation explicitly warns: “Although a reference implementation... is available from the blas directory on netlib, these routines are not expected to perform as well as a specially tuned implementation... Users are urged to determine whether such an implementation of the BLAS exists for their platform.”